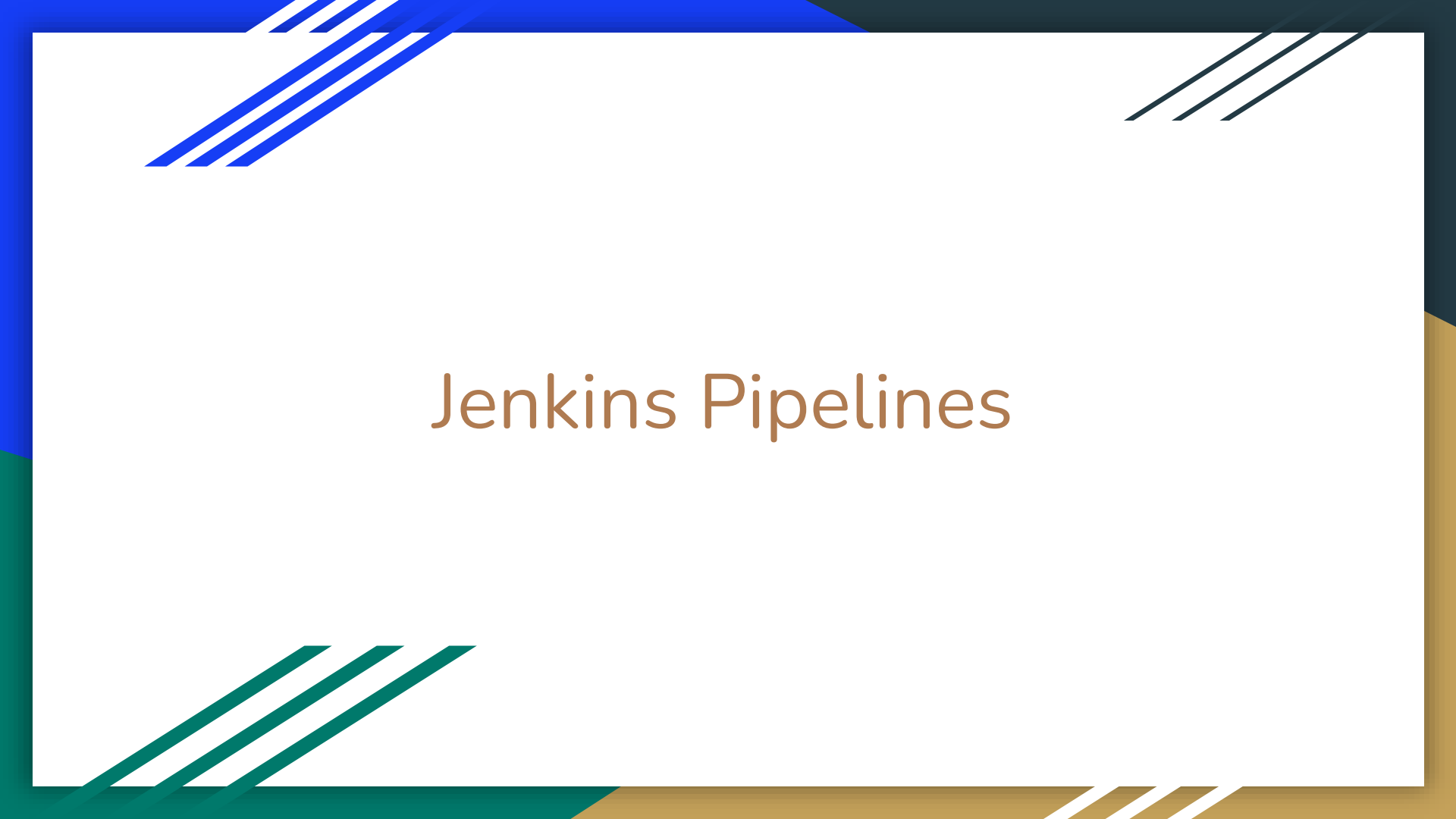




Jenkins Pipelines

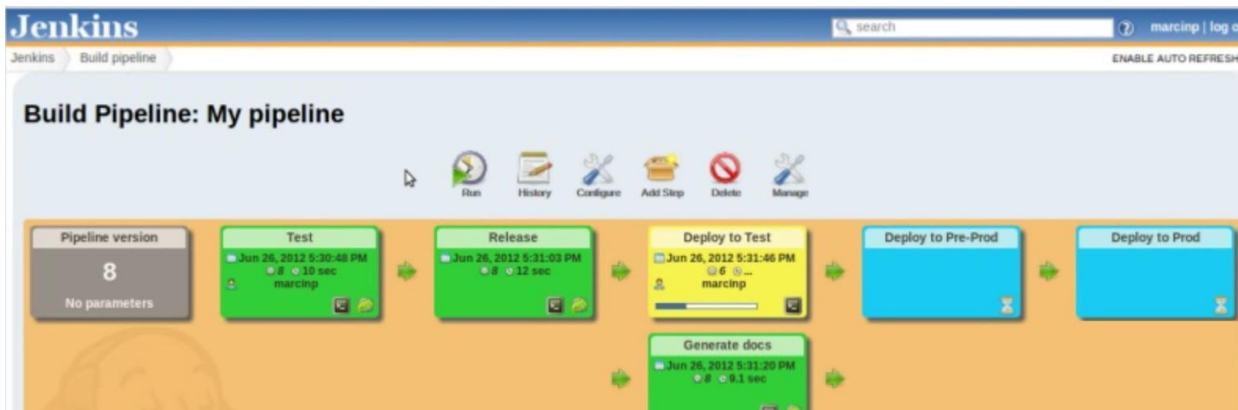
Why Jenkins

- Established open source project
- On premise installation
- Thousands of plugins
- Integrates with many tools/services
- Tailored to CI/CD



History of CI/CD with Jenkins

- Downstream Jobs
- Job A triggers job B triggers job C triggers job D trig...
- If one job fails, fail all
- Build Pipeline View



Configuration as Code

- Define jobs/pipelines as code
 - Avoid point & click • In version control
 - Can live with the application
 - Scales better
- Example: .travis.yml (from TravisCI)
- Similar to GitlabCI language: php services: - redis-server before_script: - composer install script: - ./bin/phpunit -c ... notifications: slack: ...

```
language: php

services:
- redis-server

before_script:
- composer install

script:
- ./bin/phpunit -c ...

notifications:
  slack:
...
```

Code-driven Approaches in Jenkins

- Jenkins Job Builder
 - Python / YAML based, from the OpenStack project
- Job DSL plugin
 - Groovy DSL! (e.g. query Github API and create jobs for all branches)
 - Great thing!
 - But creates single jobs `job('my-project-main') { scm { git('https://github.com/...') } triggers { scm('H/15 * * * *') } publishers { downstream('my-project-unit') } }`

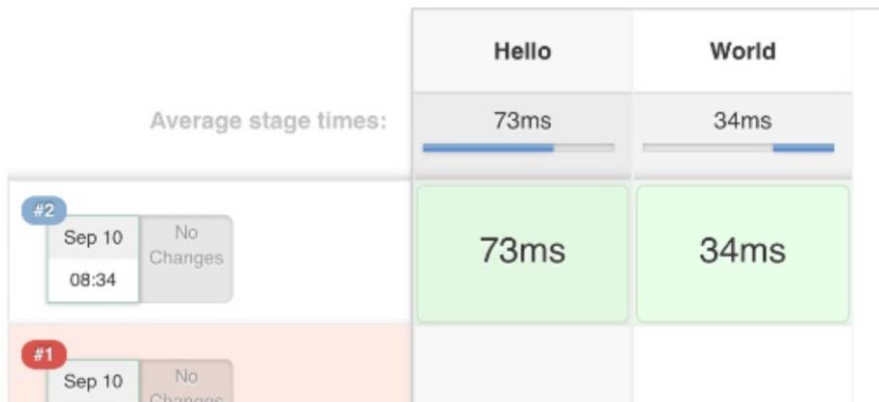
```
job('my-project-main') {  
    scm {  
        git('https://github.com/...')  
    }  
    triggers {  
        scm('H/15 * * * *')  
    }  
    publishers {  
        downstream('my-project-unit')  
    }  
}
```



Jenkins Pipeline Plugins

Jenkins Pipeline Plugins

- Whole suite of plugins (10+), open-sourced earlier this year
- Shipped with Jenkins 2.0
- Formerly commercially available by CloudBees, called Workflow
- Define pipeline as code (again Groovy DSL)



```
stage("Hello") {  
    echo "*Hello*"  
}  
stage("World") {  
    echo "*World*"  
}
```

Pipeline Example



Console Output

```
Started by user anonymous
[Pipeline] node
Running on master in /var/lib/jenkins/jobs/T3CM/jobs/Example/workspace
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] sh
[workspace] Running shell script
+ echo Could run composer install now
Could run composer install now
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Unit)
[Pipeline] sh
[workspace] Running shell script
+ echo Could run phpunit now
Could run phpunit now
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
... ..
```

```
node {
    stage("Build") {
        sh "echo Could run 'composer install' now"
    }
    stage("Unit") {
        sh "echo Could run 'phpunit' now"
    }
    // ...
}
```


Pipeline Execution

- node step allocates executor slot (“heavyweight executor”)
 - As other Jenkins jobs
 - Filter node by labels (i.e. `node('php7')`, `node('master')`)
 - Required for heavy lifting
 - Avoid many sequential allocations (slows down pipeline progress)
- Code outside of node
 - Flyweight executor
 - Running on master, “for free”
- Pipeline execution survives Jenkins restarts (CPS)

Pipeline DSL Steps

- Shellout
 - *nix systems: sh
 - `sh("make"), sh("rm -rf /")`
 - Windows systems: bat
 - `bat("C:Program Files...")`
- SCM
 - `checkout("https://github.com/..git")`
- File handling
 - `readFile("my.config")`
 - `writeFile(file: "README.md", text: "Hello World")`
 - `fileExists`

Pipeline DSL Steps (2)

- Build another job:

```
build("jobname")
build("jobname", wait: false, propagate: false)
Build(job: 'jobname', parameters: [
    [$class: 'StringParameterValue', name: 'target', value: target]
    [$class: 'ListSubversionTagsParameterValue', name: 'release', tag: release],
    [$class: 'BooleanParameterValue', name: 'update_composer', value:
    update_composer.to_boolean()]])
```

- Will not go further into detail

Pipeline DSL Steps (3)

- Copy workspace to next executor

```
stage("build") {  
    node {  
        sh("make")  
        stash("my-workspace")  
    }  
}  
stage("release") {  
    node('magic-release-tool') {  
        unstash("my-workspace")  
        sh("release")  
    }  
}
```

Pipeline DSL Steps (4)

- Specify custom environment variables

```
withEnv(['MYTOOL_HOME=/usr/local/mytool']) {  
    sh '$MYTOOL_HOME/bin/start'  
}
```

```
env.FOO = "bar"  
sh 'echo $BAR'
```

- Use variables provided by Jenkins

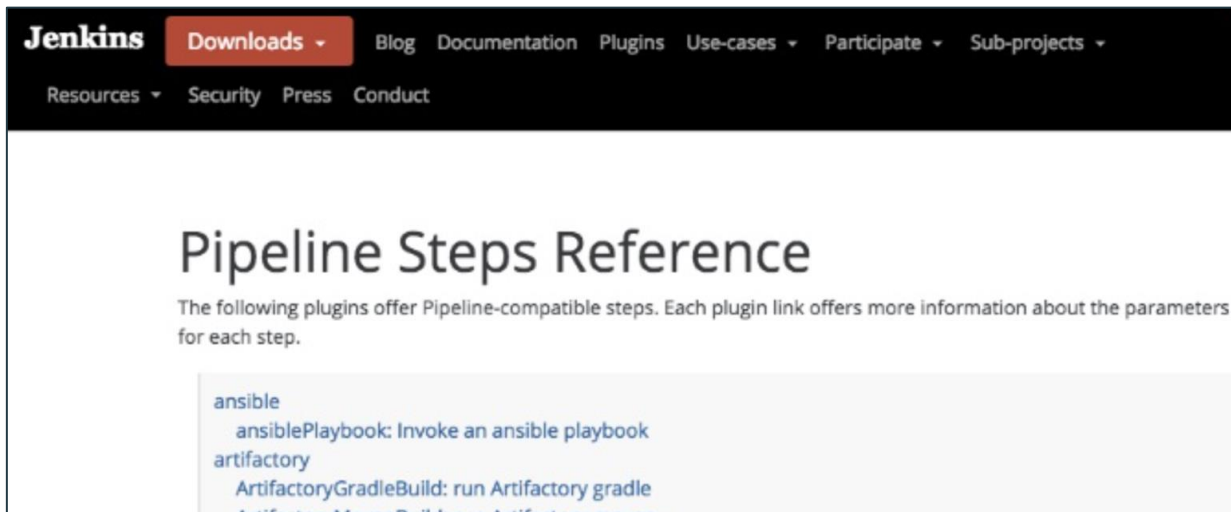
```
sh 'echo $BUILD_NUMBER > version.txt'
```

- Global Groovy variables

```
currentBuild.result = 'FAILED'
```

Even More Pipeline Steps

- Plugins contribute additional steps
- Steps available in this Jenkins instance via [Pipeline Syntax](#)
- Online reference: <https://jenkins.io/doc/pipeline/steps/>



The screenshot shows the Jenkins website's navigation bar with links for Downloads, Blog, Documentation, Plugins, Use-cases, Participate, and Sub-projects. Below the navigation bar, the page title "Pipeline Steps Reference" is displayed. A paragraph explains that the following plugins offer Pipeline-compatible steps. A list of plugins is shown, including ansible, artifactory, and artifactoryGradleBuild.

Jenkins Downloads ▾ Blog Documentation Plugins Use-cases ▾ Participate ▾ Sub-projects ▾

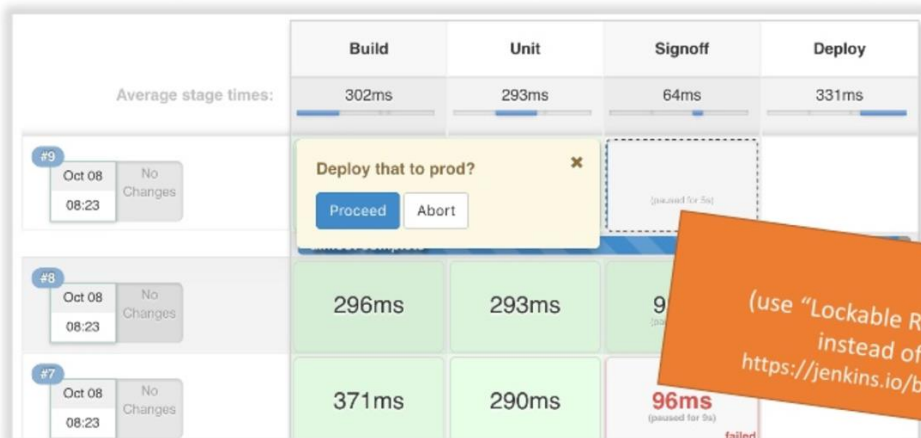
Resources ▾ Security Press Conduct

Pipeline Steps Reference

The following plugins offer Pipeline-compatible steps. Each plugin link offers more information about the parameters for each step.

- [ansible](#)
ansiblePlaybook: Invoke an ansible playbook
- [artifactory](#)
ArtifactoryGradleBuild: run Artifactory gradle

Example: Manually Confirm Deployment



Deprecated!
(use "Lockable Resources" plugin and its lock step
instead of concurrency parameter)
<https://jenkins.io/blog/2016/10/16/stage-lock-milestone/>

```
stage("Signoff") {  
    input("Deploy that to prod?")  
    milestone()  
}  
stage(name: "Deploy", concurrency: 1) { node {..} }
```

Docker

- Run build jobs within Docker containers
- No need to install software on Jenkins master/slave
- Use multiple versions of the same tool
- Containers can be existing ones or built on demand
- .. and Kubernetes

```
stage("run in docker") {  
    node {  
        withDockerContainer("php:7-fpm") {  
            sh "php -v"  
        }  
    }  
}
```


Snipped Editor & Docs

- Because first steps are hard..
- Auto-generated DSL documentation
(*Pipeline Syntax* → *Step Reference*)

Steps

Sample Step `writeFile: Write file to workspace`

File path in workspace

README.txt

Text to write

Hello World

Character encoding

Generate Groovy

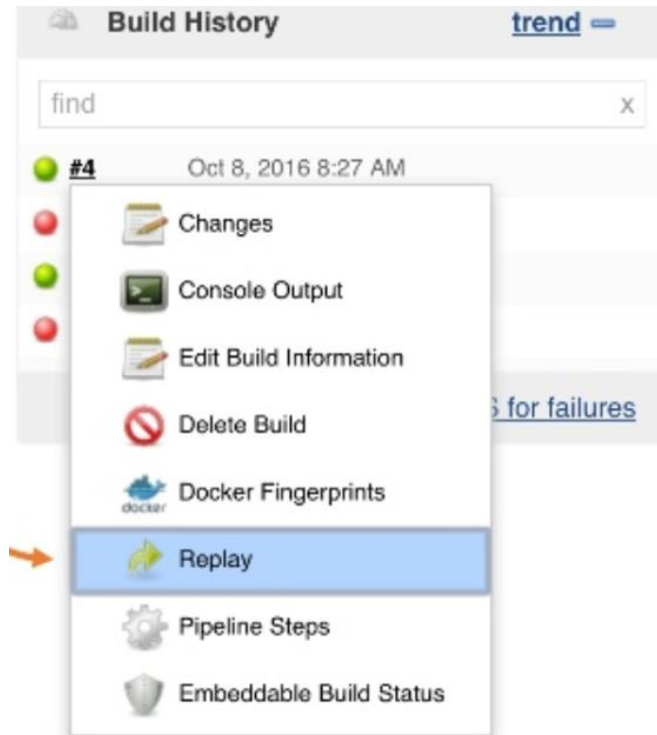
`writeFile file: 'README.txt', text: 'Hello World'`



**Where to put that
config?**

Pipeline Configuration

- Paste code into job config (fine for testing)
- Create pipelines via JobDSL
- Commit it to your repo
 - File called Jenkinsfile
 - It evolves with the application
 - It is versioned
 - Everybody can read (and modify) it
 - You can throw away your Jenkins master at any time



Multibranch & Organization Folder Plugins

- Scans a complete GitHub/Bitbucket organisation for Jenkinsfile
- Triggered by Webhook and/or runs periodically
- Automatically adds pipeline jobs per repo/branch/PR



The screenshot shows the Jenkins web interface. The top navigation bar includes 'Jenkins' and 'TYPO3's Chef Cookbooks'. The left sidebar contains links for 'Up', 'Status', 'Re-scan Organization', 'People', 'Build History', and 'GitHub'. The main content area displays the 'TYPO3's Chef Cookbooks' configuration page. It shows the folder name 'TYPO3-cookbooks' and a table of repositories. The table has columns for 'S' (Source), 'W' (Webhook), 'Repository', and 'Description'. The repositories listed are 'backuppoc', 'backuppoc = develop', 'gerrit', 'gerrit = develop', and 'gerrit = master'.

S	W	Repository ↓	Description
		backuppoc	Chef cookbook for BackupPC
		backuppoc = develop	
		gerrit	Chef cookbook for the Gerrit review system.
		gerrit = develop	
		gerrit = master	

DRY: Jenkins Global Library

- Provides shared functionality available for all jobs
- Stored on Jenkins master, available to all slaves
- Available via Jenkins-integrated Git server
- Can be loaded from remote Git repos, specified in global configuration and Jenkinsfile

```
@Library("https://github.com/..")  
node { deployToProd() }
```



Blue Ocean & Other News

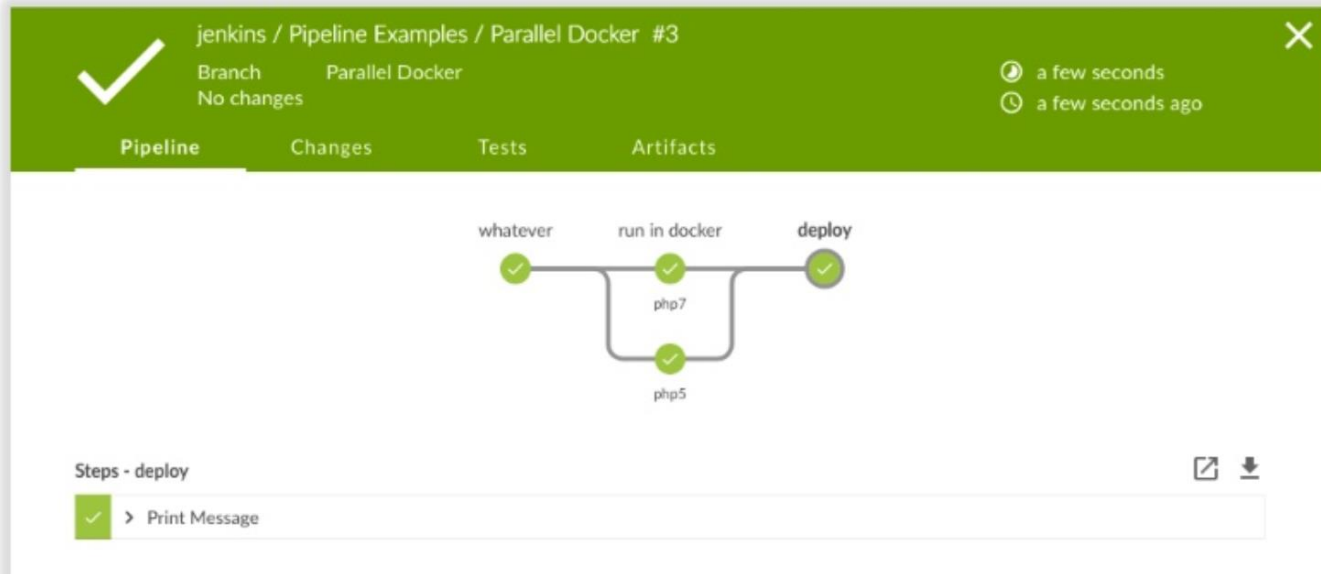
Blue Ocean

- Tailored to the pipeline plugins

Pipelines Administration			
Dashboard New Pipeline			
Name	Health	Branches	Pull Requ...
jenkins / chef-repo		-	-
jenkins / chef-repo-seed		-	-
jenkins / Pipeline Examples / Docker		-	-
jenkins / Pipeline Examples / Example		-	-
jenkins / Pipeline Examples / Hello World		-	-
jenkins / Pipeline Examples / Manual Deploy		-	-

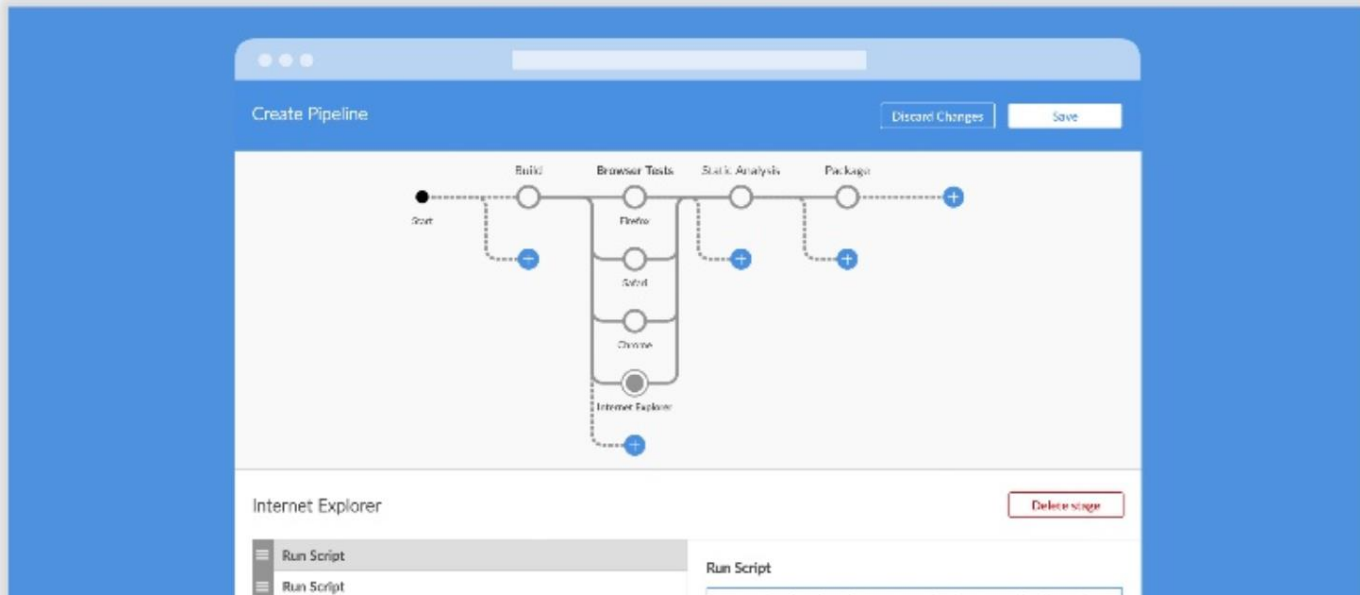
Blue Ocean

- Tailored to the pipeline plugins



Pipeline Editor*

* Recently announced, not yet available



Declarative Pipelines

- Declarative approach solves some issues (failure handling, post- build steps)

```
pipeline {
  agent docker:'node:6.3'
  stages {
    stage('build') {
      sh '..'
    }
    stage ('test') {
      sh 'npm test'
    }
  }
  postBuild {
    always {
      sh 'echo "This will always run"'
    }
    failure {
      sh 'echo "This will run only if failed"'
    }
  }
}
```

Summary

- Jenkins pipeline suite modernizes its CI/CD capabilities
- CloudBees, Inc. is very actively pushing development
- Many Jenkins plugins already compatible
- Pipeline defined as code
- Versioned
- Doesn't mess up Jenkins jobs
- Code sharing
- Automated pipeline creation based on GitHub/Bitbucket APIs
- Blue Ocean refreshes Jenkins' UI
- Still couple of rough edges (failure handling, frequent changes)